ML Challenge 2026: Business Entity Resolution Solution

Team Name: Emdash—lover
Team Members: Mehar Gilhotra, Vishwajeet Singh, Ronit Bugreja, Mankirat Singh
Submission Date: 29 September 2026

---

1. Executive Summary

We match Source 2 / Source 3 business records to each Source 1 entity with a multi-stage pipeline:

1. multilingual text normalisation, plus a native-script-to-English word dictionary learned without labels
2. two complementary candidate generators:
   - multi-index TF-IDF nearest-neighbour blocking
   - a fine-tuned multilingual-e5 **bi-encoder** whose dense nearest neighbours recover the true matches the lexical blocking misses
3. a LightGBM matcher on 106 pairwise and competition features
4. transformer cross-encoders that re-read the uncertain pairs
5. a stacker, a per-S1 expected-F0.5 decision and refinement rules

Key ideas:

- competition features (how a candidate ranks against the other candidates of the same record and of the same S1)
- test-time pseudo-labels
- a rival-aware cross-encoder
- dense retrieval with hard negatives mined from our own blocking

---

2. Methodology

2.1 Problem Analysis

- **Three countries with different conventions.**
  - India: native scripts (Devanagari, Bengali, Tamil and others) mixed with Latin transliterations; long, landmark-style addresses.
  - US: structured street addresses.
  - France: French legal forms (SARL, SAS, SCI, EURL), street words (Rue, Bd, Av., Ch.) and a small, highly repeated name vocabulary (many S1 share a name).
- **Noise patterns in the records:**
  - typos and character swaps
  - abbreviations ("Pvt Ltd" / "Private Limited", "St" / "Street", "R." / "Rue")
  - legal-form changes, word reordering, dropped tokens, added generic words

    - trade names / DBA names, website-style names
    - changed house numbers, missing addresses, transliterated names
- **Traps:**
    - near-copy decoys that differ from the S1 in one detail: a sibling word added to the name, or the house number shifted by a small amount on the same street
    - "twins": several S1 entities sharing the same name, where a record without an address cannot be told apart
- **Generator regularities (measured on train):** both countries have exactly the same singleton rate (5.58%) and the same mean number of true matches per S1 (3.46). We use them as label-free checks of the test predictions (for example, predicted matches per S1 and predicted empty rate per country, including France).
- **Test vs train:** test has about 2.8 Source-2 records per S1 against 2.3 in train. The extra records are mostly look-alike decoys, so the decision step must stay conservative.
- **Metric:** macro F0.5 per S1 weights precision twice as much as recall, and an S1 without true matches scores 1 only when nothing is predicted. Each record is assigned to at most one S1.

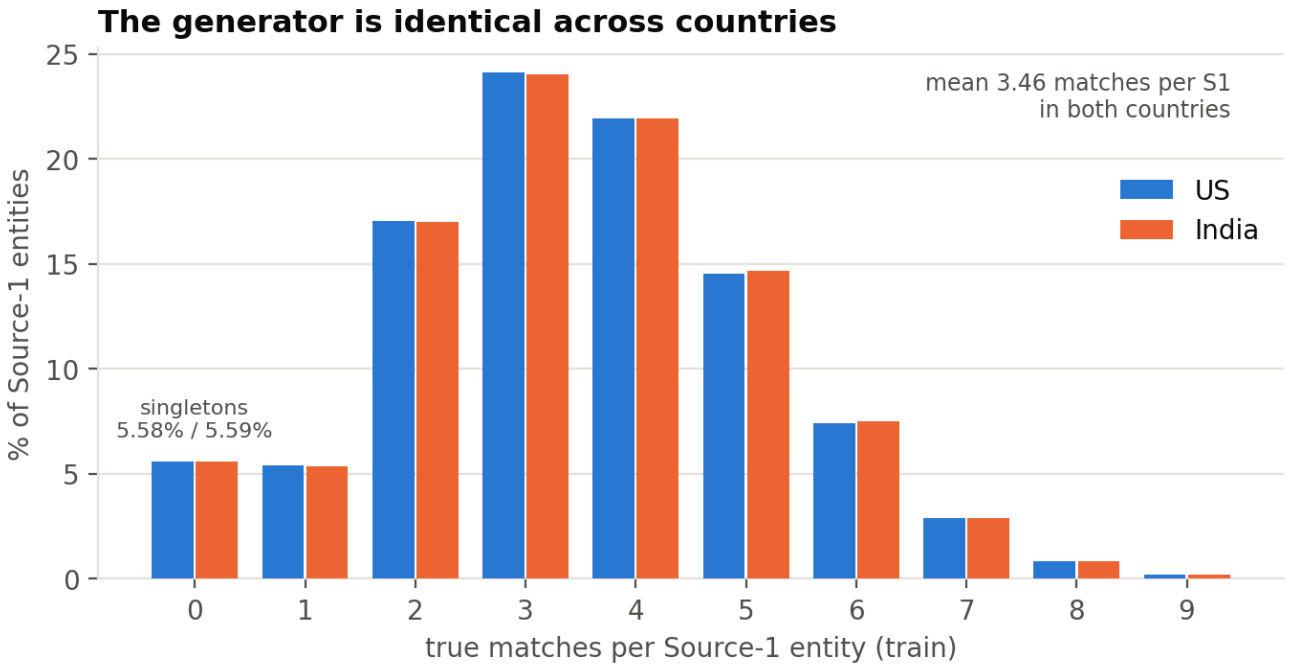


Figure: true matches per Source-1 entity in train: the same distribution in both countries

2.2 Solution Strategy

Approach Type: Hybrid: lexical blocking + dense retrieval + gradient-boosted classifier + transformer cross-encoders + stacker + expected-F0.5 decision + rule-based refinement.

Core Innovation:

- competition features over the candidate graph
- unsupervised native-script dictionary
- test-time pseudo-labelling
- a cross-encoder that sees the strongest rival S1 next to the candidate
- a contrastively fine-tuned bi-encoder used as a second, independent candidate generator, with a dedicated rescue model for the pairs only it finds

Validation: labelled train data split by region (whole states held out, never sampled at random), because random S1 sampling removes the look-alike neighbours and overstates the score. Holdouts:

- val: Tamil Nadu + DC, KS, ND, OR, UT
- val2: independent Indian states
- delhi: crowded, test-like

Every component was kept only if it improved all three holdouts under leave-one-region-out.

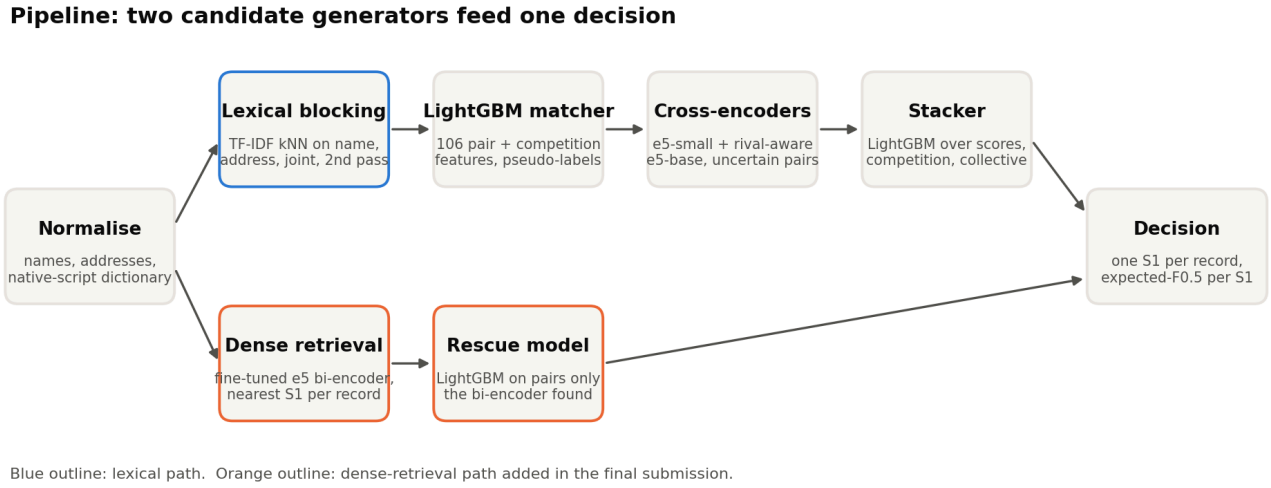


Figure: pipeline overview; the dense-retrieval path (orange) is the final addition

3. Candidate Generation (Blocking)

Blocking keys used:

1. **Lexical blocking**:
   - character / word TF-IDF nearest neighbours (per state) on the name, on the address, on name+address, and a joint name+address index
   - a quota of candidates for records without an address
   - municipal / postal code tokens and a "N°" house-number marker
   - a second pass seeded from confident matches (siblings of already-matched records)
2. **Dense retrieval (bi-encoder)**:
   - Model: intfloat/multilingual-e5-small (MIT), fine-tuned contrastively. Loss: symmetric InfoNCE, temperature 0.05, in-batch negatives plus one hard negative per pair.
   - Input text: `query: <name> | <address>` for both sides.
   - Training data: 1.74M labelled train pairs from S1s outside all holdouts, plus 0.23M confident French test pairs (pseudo-labels, final stacker p >= 0.99), so the model sees French vocabulary.
   - Hard negative: for each record, the wrong S1 with the highest lexical blocking score (the real look-alike).
   - Batches are grouped by country and state, one pair per S1 per batch. One epoch, 61 min on one RTX 3060.
   - Every record is embedded once. Its nearest S1 (cosine, same country) is added as a candidate when lexical blocking did not already propose it.

Candidate pairs generated:

- about 44.6 lexical candidates per S1 before filtering, about 77M pairs for the 1,732,544 test S1
- plus dense-retrieval candidates: 0.25-0.37 per S1 on the holdouts, 962,884 on test (0.56 per S1; test has more records per S1)
- the submitted candidate_pairs.tsv lists **7,891,249 pairs** (about 4.6 per S1): lexical pairs with LightGBM p >= 0.01, plus every dense-retrieval pair scored by the rescue model. Every match is inside it.

How we ensured true matches were not lost (measured on held-out regions):

Holdout	Lexical blocking recall	+ bi-encoder top-1	Blocking misses recovered
val	98.98%	**99.74%**	74.5%
val2	98.78%	**99.70%**	75.8%
delhi	98.47%	**99.63%**	75.8%

About 86% of the missed records that have an address are recovered: trade names, domains, native-script names, heavy typos. The remaining misses are mostly records without an address whose name is shared by many S1.

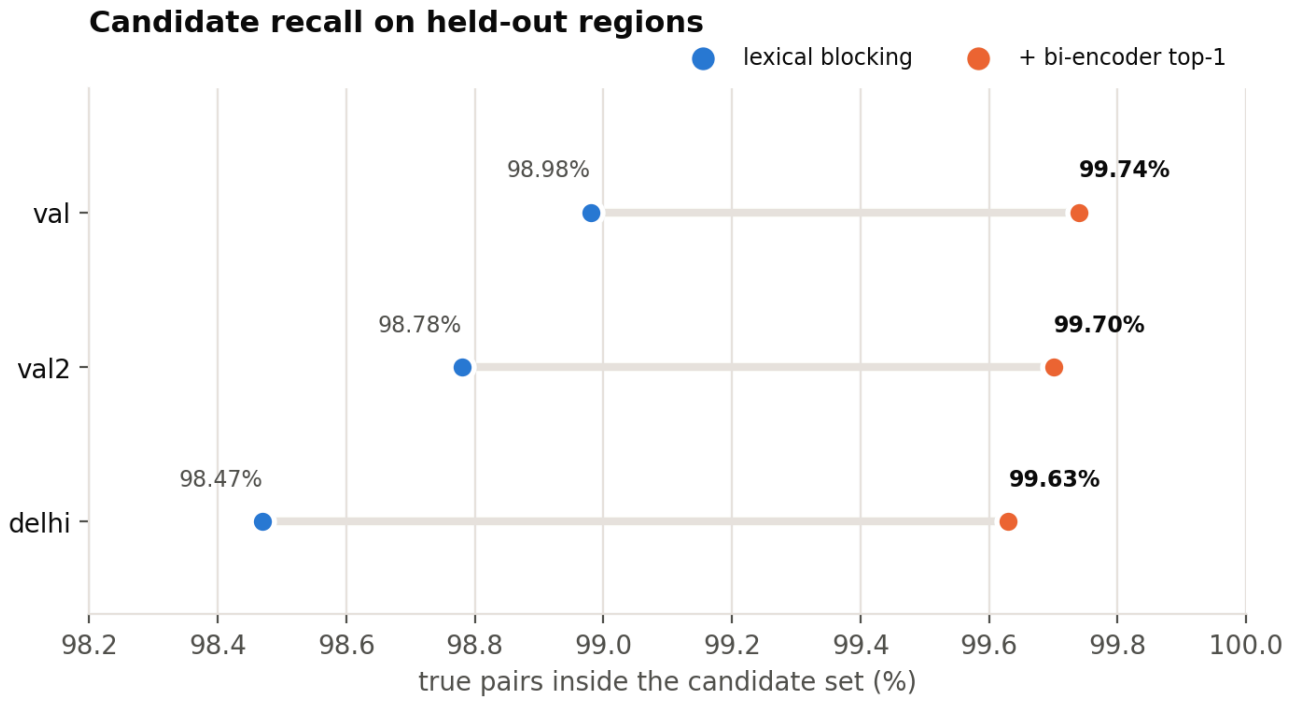


Figure: candidate recall per held-out region, lexical blocking vs. adding the bi-encoder's nearest S1

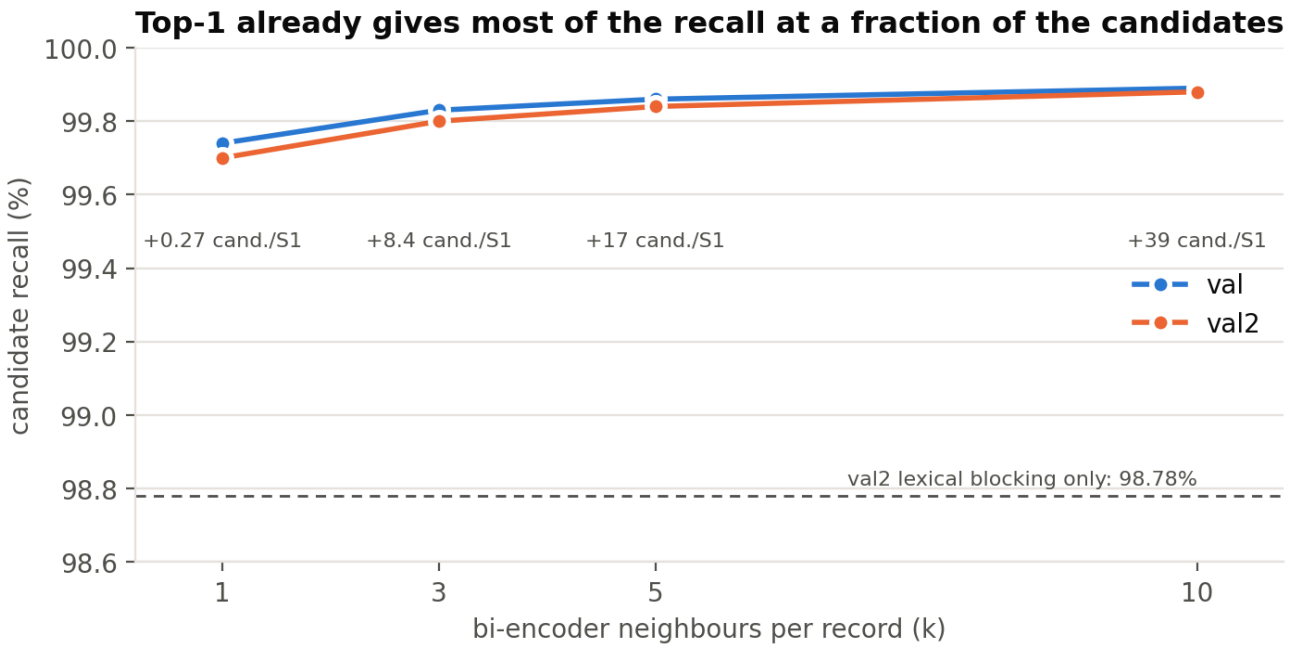


Figure: candidate recall vs. number of bi-encoder neighbours per record, with the extra candidates each adds

4. Matching Model

Features used (106 in the main matcher):

- **Name features:** token-set / partial / Jaro-Winkler ratios, IDF-weighted Jaccard and coverage, core-token and consonant-skeleton Jaccard, initials, legal-form equality / conflict, domain names, name length and token counts, transliteration flag.
- **Address features:** token-set and ratio similarity, IDF overlap, core-token Jaccard, house number equal / conflict / shared, all-number agreement and nearest numeric difference, unit equal / conflict, state equality, address-missing flag.
- **Other:**
  - which blocking index proposed the pair, plus its score and rank
  - competition features in both directions: rank, gap and margin of the pair among the record's S1 candidates and among the S1's records, per similarity measure
  - same-name counts for twins
  - collective evidence: whether the pair shares name skeleton / address / house number with the S1's other confident matches

Model type:

1. LightGBM binary classifier, trained on labelled tune pairs plus confident test pseudo-labels (p >= 0.98 positive, p <= 0.02 negative).
2. A multilingual-e5-small cross-encoder with the numeric features, on uncertain pairs.
3. A rival-aware multilingual-e5-base cross-encoder: input = record vs "candidate S1 + strongest rival S1".
4. A LightGBM stacker (31 leaves, 800 rounds) over the model scores, score-competition and collective features. It is trained on the three labelled holdouts and validated leave-one-region-out over 11 regions.

5. Rescue model for dense-retrieval pairs (new): a LightGBM scoring the pairs that only the bi-encoder proposed. Features:
    - cosine, the record's top cosine and its margin over the second S1
    - how many records retrieve the S1, and the S1's rank among them
    - name / skeleton / address fuzzy ratios, shared house numbers and numbers, state agreement, address-missing and domain flags, source
    - the record's best score in the main pipeline and the S1's current matches Leave-one-region-out AUC 0.988; precision 0.98 at p >= 0.9.

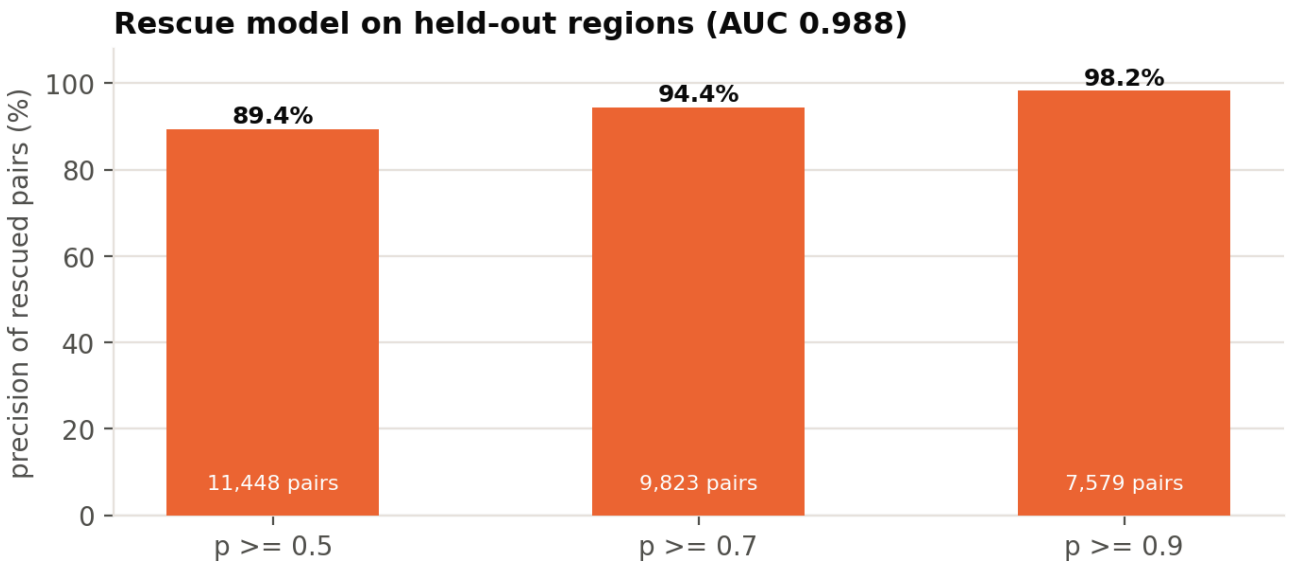


Figure: precision of rescued pairs on the held-out regions at three score thresholds

Decision (threshold selection):

- Main scores and rescue scores are merged, and each record keeps only its best S1.
- For each S1, the candidate set that maximises the **expected F0.5** is chosen: Monte Carlo over the calibrated probabilities, including the empty set. This replaces a global threshold.
- Refinement rules follow: trap correction, same-script links, source cap, a contested no-address rule. Each rule was kept only if it improved the holdouts.

---

5. Results & Error Analysis

F_0.5 Score (macro), holdouts, leave-one-region-out:

	val	**val2**	**delhi**
Previous pipeline (leaderboard 0.987495)	0.99129	0.99197	0.99036
+ bi-encoder dense retrieval and rescue (final)	**0.99357**	**0.99394**	**0.99302**
Gain	+0.00228	+0.00196	+0.00266

The largest gain is on the crowded delhi holdout, the one most similar to test.

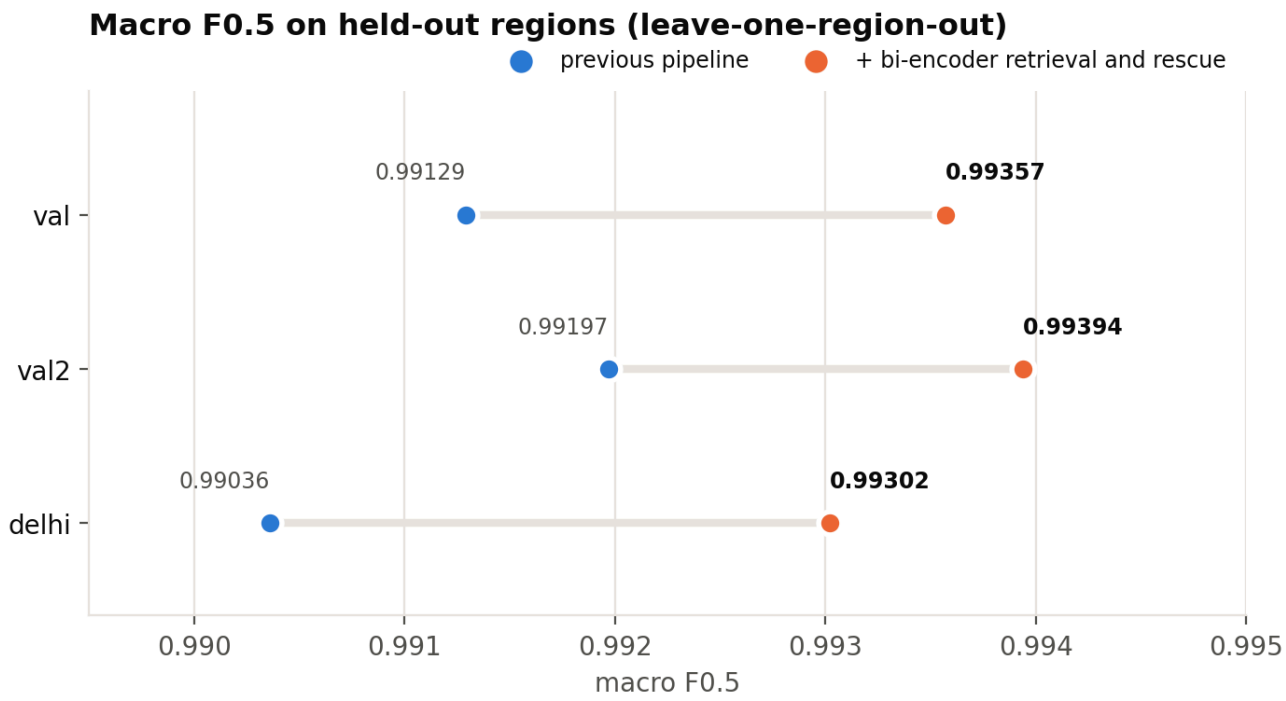


Figure: macro F0.5 per held-out region before and after dense retrieval and rescue

On test:

- 5,877,096 matches (previous pipeline: 5,828,923); 48,213 of them come from dense-retrieval rescue.
- Predicted matches per S1 moved toward the generator's 3.46: India 3.351 -> 3.385, France 3.354 -> 3.415, US 3.385 -> 3.392.
- Empty-S1 rates stay near the generator's 5.58% singleton rate: US 5.72%, India 5.71%, France 5.40%.

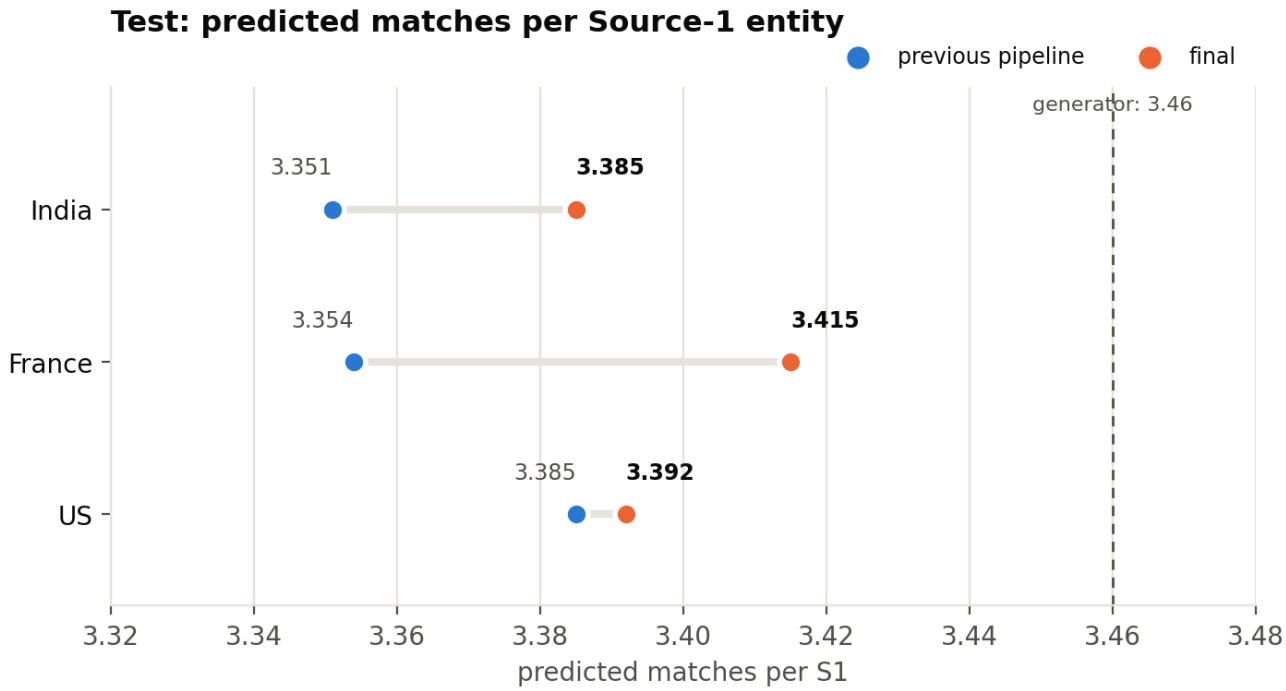


Figure: predicted matches per Source-1 entity on test, per country, against the generator's 3.46

Common false positives (wrong merges):

- near-copy decoys that share the S1's name and street but add a sibling word or change the house number

- twins without an address, where the model picks the wrong one of several same-name S1

Common false negatives (missed matches):

- records without an address whose name is shared by several S1 (not separable from the text)
- true pairs with perturbed house numbers that look like decoys
- before the bi-encoder: trade names, website-style names and native-script names with short addresses, which lexical blocking could not reach

---

6. Conclusion

Lexical multi-index blocking, a feature-rich LightGBM matcher with competition features and transformer cross-encoders took the leaderboard score from 0.952 to 0.987495. The final step adds a contrastively fine-tuned multilingual-e5 bi-encoder as a second candidate generator, with a rescue model and an expected-F0.5 decision. It recovers three quarters of the matches that lexical blocking missed and raises every held-out region by 0.002 to 0.0027. The main remaining limit is records without an address whose name is shared by several S1 entities.

Appendix

A. Code Artefacts

The code is in the submission zip under `code/business_entity_resolution/`: `src/`, `configs/`, `README.md`, `requirements.txt`.

- Entry point: `src/run_pipeline.py --stage <stage>` for the main pipeline, then `src/biencoder.py` (train / embed / retrieve) and `src/final.py` (stacker, rescue, decision, output files).
- The README lists the full command order that reproduces `output/matching_results.tsv` and `output/candidate_pairs.tsv` from the provided data.

B. Additional Results

Submission	Leaderboard	Main change
1	0.952	first full pipeline
2	0.962	IDF and competition features, blocking fixes
3	0.975	small cross-encoder + stacker
5	0.97670	refinements
7	0.983844	native dictionary, new blocking, pseudo-labels
8	0.984428	rival-aware e5-base cross-encoder
9	0.987495	stacker on three holdouts, expected-F0.5 decision, collective features + bi-encoder dense retrieval + rescue model

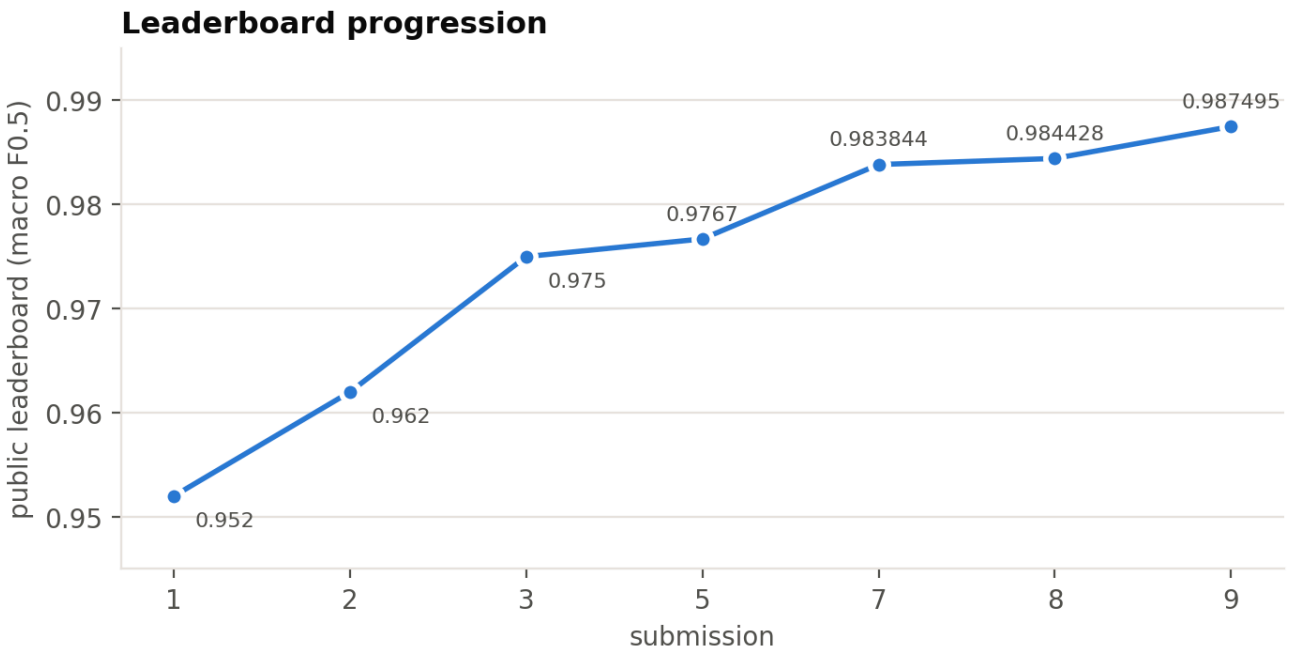


Figure: public leaderboard score by submission

Models used, all open-licensed and under 8B parameters:

- intfloat/multilingual-e5-small (MIT)
- intfloat/multilingual-e5-base (MIT)
- LightGBM (MIT)

No external data, APIs or lookups were used.